

Informe 2

Proyecto de Colaboración en GitHub

Nombres: Fernando Palma Olea y Isidora Salgado Camus

Asignatura: Introducción a los Repositorios de Código Distribuido

Fecha: 24 de Julio de 2026

Repositorio Fernando Palma: <https://github.com/fernaaando/proyecto-repo>

Repositorio Isidora Salgado: <https://github.com/printf-scarlettqrow/OptimizerProyect>

Pull Request Fernando Palma: <https://github.com/printf-scarlettqrow/OptimizerProyect/pull/2>

Pull Request Isidora Salgado: <https://github.com/fernaaando/proyecto-repo/pull/2>

Descripción de los proyectos originales

En esta sección se detallan los dos repositorios originales del trabajo colaborativo: el **Clasificador Astronómico**, desarrollado por Fernando Palma y el **OptimizerProject**, perteneciente a Isidora Salgado.

Objetivo de cada proyecto

Clasificador Astronómico: El principal objetivo de este proyecto es desarrollar e implementar un modelo de clasificación capaz de procesar y clasificar de manera automática cuerpos celestes (estrellas, galaxias, cuásares), basándose solo en sus datos espectrales, usando un clasificador Random Forest.

OptimizerProject: El objetivo de este proyecto es desarrollar un software que calcule la ruta más corta entre múltiples destinos de reparto, utilizando algoritmos de optimización y visualizaciones interactivas para facilitar el análisis de datos espaciales.

Funcionalidades principales

Clasificador Astronómico: Las funcionalidades de este proyecto abarcan desde un análisis exploratorio de datos (EDA) para procesar y visualizar las características de luz para luego, implementar y entrenar dos modelos predictivos, Random Forest y KNN, para categorizar automáticamente galaxias, estrellas y cuásares. Finalmente, el proceso cierra con la evaluación de rendimiento de ambos modelos para validar que el modelo definitivo alcance una alta precisión.

OptimizerProject: Las funcionalidades de este proyecto abarcan desde el cálculo de rutas óptimas de reparto resolviendo el Problema del Viajante (TSP), utilizando los algoritmos de vecino más cercano y 2-opt, para luego generar mapas interactivos (en HTML) y gráficos estadísticos que analizan las métricas de entrega y el ahorro de distancias logrado.

Estado de los repositorios antes de la colaboración

Clasificador Astronómico: El repositorio antes de la colaboración se encontraba todo en su rama principal main, tenía un entorno configurado usando un archivo `.gitignore` y un `README.md` y el código se ubicaba en el archivo `clasificador.ipynb`. A nivel funcional el proyecto ya incluía el procesamiento de datos (EDA) y la implementación del modelo predictivo *Random Forest*, habiendo descartado y resuelto los conflictos con modelos secundarios.

OptimizerProject: El repositorio se encontraba completamente integrado en su rama local main, con un entorno configurado mediante los archivos `.gitignore`, `README.md` y `requirements.txt`. El proyecto presentaba una estructura organizada en carpetas específicas para los datos (`data/`), el análisis exploratorio ya finalizado (`notebooks/`), el núcleo del código fuente (`src/`) y las pruebas (`tests/`). Funcionalmente, el código ya ejecutaba el

procesamiento de datos, los algoritmos de optimización de rutas y la generación de mapas interactivos.

Organización del trabajo

Integrantes de la pareja: Conformado por Fernando Palma e Isidora Salgado, en los cuales, ambos actuaron con doble rol, ambos fueron mantenedores de su propio repositorio original de la primera entrega y también fueron colaboradores, en la que cada uno operó como desarrollador externo del repositorio de su compañero de trabajo.

Contribución asignada a cada integrante

Fernando en proyecto de Isidora: La contribución que se hizo en el repositorio de Isidora fue la implementación de una función para exportar los resultados en un formato .JSON y un archivo con pruebas de test para garantizar su correcto funcionamiento.

Isidora en proyecto de Fernando: La contribución que se realizó en el repositorio de Fernando consistió en implementar una función que, utilizando el modelo ya entrenado, permite ingresar los parámetros de un objeto astronómico nuevo para obtener su clasificación (galaxia, cuásar o estrella) junto con las probabilidades asociadas a cada clase.

Forma en que se coordinaron las actividades

La coordinación del proyecto se realizó de manera separada, comenzando con la apertura de un *issue* para sugerir cambios. Luego, el desarrollo avanzó a través de un *fork* y una rama exclusiva para las características a agregar. Posteriormente se abrió un *Pull Request* en el que, al estar de acuerdo con los cambios, estos se aceptaron y se integraron mediante un *merge* a la rama *main* de cada proyecto.

Issues creados

Issue de Fernando en el repositorio de Isidora

Descripción de la propuesta: Desarrollar una función que exporte los resultados de las rutas calculadas a un archivo de formato .JSON, además, se realizaron pruebas unitarias utilizando pytest.

Justificación de la mejora: El sistema antes de esta integración solo mostraba los resultados de las rutas calculadas de forma visual pero no tenía una alternativa para exportar los datos en crudo por si luego querían ser procesados por otro sistema.

Criterios de aceptación: Para aceptar la mejora el sistema de Isidora debe seguir funcionando de la misma manera solo que ahora luego de ejecutar el programa debe

generar un archivo **.JSON** con las rutas calculadas y también se debe incluir un archivo con test sin errores.

Respuesta del mantenedor: Isidora revisó el *issue*, verificó que la propuesta tuviera valor y lo aceptó, quedando como mantenedora a cargo de revisar los cambios.

Issue de Isidora en el repositorio de Fernando

Descripción de la propuesta: Se propone agregar la función **clasificar_objeto**, que usa el modelo ya entrenado para clasificar un objeto astronómico nuevo (*GALAXY*, *QSO* o *STAR*) y entregar las probabilidades por clase de que sea o no ese objeto.

Justificación de la mejora: El modelo solo se usaba para evaluar el dataset de prueba; esta función permite aplicarlo a nuevos datos ingresados manualmente, con validación de entradas y documentación interna que facilita su uso por el equipo.

Criterios de aceptación: Para aceptar la mejora, el sistema de Fernando debe seguir funcionando correctamente, solo que ahora debe incluir la función **clasificar_objeto**, que reciba los parámetros correspondientes (**u**, **g**, **r**, **i**, **z** y **redshift**), y las probabilidades de que el objeto corresponda a cada cuerpo celeste (*GALAXY*, *QSO* o *STAR*).

Respuesta del mantenedor: Fernando evaluó el *issue*, verificó la pertinencia de la propuesta y la aceptó. Asumiendo el rol de mantenedor responsable de la revisión de los cambios.

Proceso de desarrollo

Creación del *fork* y configuración de *origin* y *upstream*

En ambos casos, el ciclo de desarrollo se llevó a cabo de la misma manera. Primero se realizó un *fork* del repositorio original hacia la cuenta personal de cada colaborador, y luego se procedió a clonar el repositorio de forma local para poder trabajar. Posteriormente, se configuraron los repositorios remotos, manteniendo el enlace del *fork* como *origin* y agregando la URL del repositorio del mantenedor como *upstream*.

Desarrollo de Fernando en el repositorio de Isidora

Creación de la rama: Se creó una rama aparte de la rama principal *main*, llamada *feature/exportar-json*.

Commits realizados: El desarrollo se llevó a cabo en 3 *commits* totales, el primer *commit* se basó en la creación de la función para exportar los resultados a un formato **.JSON** modificando los archivos **src/controller.py** y **src/dataUtils.py**. El segundo *commit*, se enfocó en la integración de la función creada en el archivo **controller.py** y finalmente,

el tercer *commit* fue básicamente para agregar la contextualización de los cambios en el README.md.

Sincronización con el repositorio original: Para subir la implementación primero se ejecutó `git fetch upstream` para luego realizar un *merge* entre *upstream/main* y la rama *main* local. Luego de comprobar que el estado era *Already up to date*, se hizo el *merge* final con la rama *feature/exportar-json*, teniendo así cero conflictos antes de hacer `git push origin`.

Desarrollo de Isidora en el repositorio de Fernando

Creación de la rama: Se crea una nueva rama, siguiendo los lineamientos de la rama *main*, llamada *feature/clasificar-objeto-nuevo*.

Commits realizados: El desarrollo de la contribución se estructuró en tres *commits*. El primero consistió en la implementación de la función `clasificar_objeto`, incorporando un filtro para verificar que los parámetros ingresados correspondan al tipo de dato esperado antes de su procesamiento. El segundo *commit* se enfocó en agregar las pruebas de esta nueva función utilizando casos reales correspondientes a cada clase. Finalmente, el tercer *commit* actualizó la documentación para explicar la sección de clasificación de nuevos objetos.

Sincronización con el repositorio original: Previo a la subida de la implementación, se ejecutó `git fetch upstream` para sincronizar los eventuales cambios mediante un *merge* entre *upstream/main* y la rama *main* local. Tras comprobar que el estado arrojaba *Already up to date*, se procedió a actualizar la rama *feature/clasificar-objeto-nuevo*, paso que garantizó la ausencia de conflictos al momento de ejecutar `git push origin`.

Pull Request

Descripción de los cambios implementados y archivos modificados

Contribución de Fernando en Optimizer: Se abrió un *Pull Request* en el repositorio de Isidora que incorpora la función de exportación de rutas en formato `.JSON` y el archivo con las pruebas automatizadas con `pytest`. Los archivos modificados y creados fueron `src/controller.py`, `src/dataUtils.py`, `test_export_json.py` y `README.md`.

Contribución de Isidora en Clasificador: Se generó un *Pull Request* para el repositorio de Fernando, incorporando al final del archivo `clasificador.ipynb` la función para clasificar nuevos objetos. Esta mejora incluyó pruebas específicas para verificar el correcto funcionamiento del código, además de las actualizaciones correspondientes en la documentación del archivo `README.md`. De esta forma, los archivos modificados fueron: `clasificador.ipynb` y el `README.md`.

Relación entre el *Pull Request* y el *issue* y forma utilizada para aprobar los cambios

Cada *Pull Request* se vinculó directamente con el *issue* que generó esta implementación, utilizando la sintaxis de *GitHub*, *Closes #1*, para resolver todos los aspectos de aceptación planteados al inicio.

Para aprobar los cambios, se hizo lo siguiente:

- **Isidora**, en su rol de mantenedora de su repositorio, ejecutó el archivo con las pruebas utilizando **pytest** (**pytest test_export_json.py**) y luego verificó que el archivo **.JSON** generado tuviera la estructura de datos correcta para las rutas.
- **Fernando**, en su rol de mantenedor de su repositorio, ejecutó el archivo **clasificador.ipynb** y verificó que las pruebas se ejecutaran sin errores. Además de esto, revisó el código a modo general para verificar la consistencia de los cambios implementados.

Proceso de revisión

Comentarios realizados por el mantenedor y cambios solicitados

- En su rol de mantenedora del repositorio OptimizerProyect, **Isidora** revisó el *Pull Request* de **Fernando** y, mediante la sección de comentarios de la solicitud, requirió incorporar un bloque de excepciones para el manejo de errores en las rutas de los archivos, junto con la creación de una prueba unitaria utilizando **pytest**.
- **Fernando** por su parte, revisó el *Pull Request* de **Isidora** y registró sus observaciones directamente en los comentarios del requerimiento. En primer lugar, solicitó incluir un *docstring* en la función **clasificar_objeto** para documentar internamente el diccionario de retorno y los parámetros recibidos. Además, requirió modificar el formato de salida de las probabilidades, que originalmente se retornaban como un tipo de dato de **NumPy**, con el propósito de optimizar la legibilidad de los resultados.

Correcciones realizadas por el colaborador

- **Fernando** implementó las correcciones solicitadas en el repositorio OptimizerProyect, incorporando el manejo de errores en la función de exportación **.JSON** y añadiendo el archivo de pruebas automatizadas. Tras informar la resolución en los comentarios, **Isidora** verificó los cambios y aprobó el *Pull Request* mediante la opción *Approve*.

- **Isidora** resolvió los requerimientos levantados por **Fernando** mediante una nueva actualización en la rama trabajada. En ella, se integró el *docstring* para detallar los parámetros de la función y modificó la presentación de los resultados finales, reemplazando el tipo de dato **NumPy** por un formato numérico normal. Una vez realizadas estas modificaciones, **Fernando** revisó los cambios, verificó el cumplimiento técnico de lo solicitado y procedió a aprobar el *Pull Request* mediante la opción *Approve*.

Respuestas a los comentarios

Todos los comentarios, solicitudes de cambio y respuestas están en el **Anexo A** y **Anexo B**, en el orden en que ocurrieron.

Integración de las contribuciones

Una vez que ambas solicitudes de cambios fueron resueltas y los *Pull Request* obtuvieron la aprobación final, los mantenedores de cada repositorio procedieron a integrar el código mediante un *merge* hacia la rama principal:

- En el repositorio *OptimizerProject*, **Isidora** integró la rama **feature/exportar-json** de **Fernando**.
- En el repositorio *Clasificador Astronómico*, **Fernando** realizó el *merge* de la rama de **Isidora**, **feature/clasificar-objeto-nuevo**.

Cierre de Issues y sincronización final

Una vez finalizadas las integraciones en cada repositorio, los *issues* vinculados a las tareas se cerraron. Como paso final, ambos desarrolladores ejecutaron una última actualización en sus entornos locales mediante el comando **git pull upstream main** para sincronizar los repositorios.

Análisis del historial

OptimizerProject

En el repositorio *OptimizerProject*, el análisis de historial busca encontrar la integración de la funcionalidad de exportación **.JSON**. Respecto de los *commits*, **Fernando** ejecutó cinco *commits* en la rama **feature/exportar-json**, mientras que **Isidora** en su rol de mantenedora, registra el *commit* de fusión generado al aprobar el *Pull Request*. El detalle de la intervención externa, junto con los archivos modificados, es el siguiente:

- **Commit 1:** Se crea la función base para implementar la exportación **.JSON**, modificando el archivo **src/dataUtils.py**.
- **Commit 2:** Integra la función desarrollada para exportar la ruta calculada, editando el archivo **src/controller.py**.
- **Commit 3:** Actualiza la documentación del proyecto, modificando exclusivamente el archivo **README.md**.
- **Commit 4:** Aplica correcciones a errores detectados en la sintaxis de retorno, editando nuevamente el archivo **src/controller.py**.
- **Commit 5:** Incorpora el manejo de errores en **src/controller.py** y agrega el nuevo script de pruebas unitarias para la exportación **.JSON**.

Antes de la integración, el historial de *main* no contenía referencias a la exportación **.JSON** ni a los archivos **src/dataUtils.py** o el script de pruebas asociado. Tras la fusión del *Pull Request*, estos cinco commits se incorporan de forma íntegra, agregando la nueva funcionalidad junto con su documentación. A nivel gráfico, esto se observa como una ruta de desarrollo paralela que termina uniéndose a *main*. La autoría de esta contribución queda clara al verificar el usuario y correo de **Fernando** en los nuevos *commits*, lo cual se comprueba en la captura obtenida con el comando **git log --pretty=format:"%h | %ad | %an | %ae | %s" --date=short (Ver Anexos)**. Por último, la visualización de ramas confirma la separación del código y su unificación final, tal como se expone en la captura generada mediante **git log --graph --oneline --all (Ver Anexos)**.

Clasificador de Objetos Astronómicos

En el repositorio del Clasificador de Objetos Astronómicos, el historial muestra la incorporación de la función de clasificación. **Isidora** efectuó cuatro *commits* en la rama **feature/clasificar-objeto-nuevo**, finalizando con el commit de integración ejecutado por **Fernando** en su rol de mantenedor. Las modificaciones se distribuyen así:

- **Commit 1:** Se agrega la función **clasificar_objeto** incluyendo la validación de los tipos de entrada, modificando el archivo **clasificador.ipynb**.
- **Commit 2:** Incorpora pruebas de la nueva función utilizando casos reales de cada clase, editando nuevamente el archivo **clasificador.ipynb**.
- **Commit 3:** Actualiza la documentación del proyecto, modificando exclusivamente el archivo **README.md**.

- **Commit 4:** Aplica las correcciones de la revisión para mejorar la legibilidad de las probabilidades y agregar el *docstring*, editando nuevamente el archivo **clasificador.ipynb**.

Antes de la integración, el historial de *main* no contenía la función para clasificar nuevos objetos, ni las validaciones o pruebas con casos reales dentro del archivo **clasificador.ipynb**. Tras la fusión del *Pull Request*, estos cuatro *commits* se incorporan de forma íntegra, agregando la nueva capacidad predictiva junto con su respectiva documentación. A nivel gráfico, esto se observa como una ruta de desarrollo paralela que termina uniéndose a *main*. La autoría de esta contribución queda clara al verificar el usuario y correo de **Isidora** en los nuevos commits, lo cual se comprueba en la captura obtenida con el comando `git log --pretty=format:"%h | %ad | %an | %ae | %s" --date=short (Ver Anexos)`. Por último, la visualización de ramas confirma la separación temporal del código y su unificación final, tal como se expone en la captura generada mediante `git log --graph --oneline --all (Ver Anexos)`.

Reflexión individual

Reflexión de Fernando Palma

1. ¿Qué dificultades encontró al contribuir a un repositorio ajeno?

La principal dificultad fue la de entender la lógica de los algoritmos de optimización de ruta que no escribí desde cero, requirió tiempo de lectura y análisis, fue un desafío identificar en qué parte del flujo debía intervenir para extraer los datos y generar la exportación JSON.

2. ¿Qué aprendió al revisar el código de otra persona?

Aprendí la importancia de aplicar una perspectiva externa al código, observar cómo se implementaron las validaciones explícitas para dar un formato a la salida me demostró formas más limpias y seguras de manejar entradas en Python.

3. ¿Qué aprendió al recibir comentarios sobre su propio proyecto?

Recibir el *issue* inicial en el clasificador astronómico fue muy revelador, me hizo darme cuenta de que al enfocarme tanto en el entrenamiento del Random Forest y alcanzar un 98% de precisión, descuidé la usabilidad práctica del modelo. Recibir estos comentarios me ayudó a entender que un proyecto de software no termina cuando el código compila o el modelo entrena, sino que cuando realmente es útil.

4. ¿Qué diferencia existe entre realizar un merge local y aceptar un *Pull Request*?

Un merge local es una acción directa que une historiales en mi propio computador, en cambio, aceptar un *Pull Request* en GitHub es una acción colaborativa, es como un hilo en el que se expone, se discute, se revisa línea a línea y se aprueba antes de integrarse en la rama principal.

5. ¿Qué ventajas y dificultades presenta el flujo basado en *forks* y *Pull Requests*?

La principal ventaja es la seguridad y el aislamiento ya que, la rama main del repositorio original se mantiene totalmente protegida. La dificultad se basa principalmente en la sobrecarga administrativa que esto añade, ya que se requiere orden para mantener ambos remotos, gestionar ramas y estar atento a los cambios para evitar conflictos.

Reflexión de Isidora Salgado

1. ¿Qué dificultades encontró al contribuir a un repositorio ajeno?

La principal dificultad estuvo relacionada en el manejo de comandos específicos para operar entre un repositorio bifurcado (*fork*) y el repositorio original (*upstream*), así como en la comprensión técnica de sus diferencias. También, representó un desafío importante identificar áreas de mejora en un código de otro autor, debido a la necesidad de interiorizarse previamente con su lógica y estructura de desarrollo.

2. ¿Qué aprendió al revisar el código de otra persona?

Este proceso me permitió comprender los diferentes enfoques lógicos que un desarrollador puede aplicar para resolver un problema. Se observó que, aunque las estructuras de pensamiento generales pueden ser similares, existen variaciones importantes en las implementaciones específicas. Además, se evidenció la importancia de la documentación y los comentarios, ya que el nivel de explicitud requerido para comprender un bloque de código varía significativamente de un individuo a otro.

3. ¿Qué aprendió al recibir comentarios sobre su propio proyecto?

Valoro positivamente la retroalimentación recibida a través de los *issues*, ya que las sugerencias evidenciaron perspectivas de mejora no contempladas inicialmente. En particular, la solicitud de integrar un formato de exportación **.JSON** demostró ser una funcionalidad clave para la escalabilidad del proyecto, pues este tipo de implementaciones facilita la compatibilidad y permite que los datos generados sean procesados por futuros modelos o herramientas de manera más eficiente y estandarizada.

4. ¿Qué diferencia existe entre realizar un *merge* local y aceptar un *Pull Request*?

Básicamente, un *merge* local es una operación directa que fusiona ramas exclusivamente dentro del entorno de desarrollo personal, sin requerir validación externa. Por el contrario, aceptar un *Pull Request* es un proceso diseñado para entornos de trabajo colaborativos, implica una etapa previa y estructurada de revisión de código por parte de otros desarrolladores, lo cual fomenta la retroalimentación, la detección temprana de errores y la discusión técnica antes de integrar definitivamente los cambios a la rama principal.

5. ¿Qué ventajas y dificultades presenta el flujo basado en *forks* y *Pull Requests*?

La principal ventaja de este flujo radica en la protección de la rama principal (*main*) del repositorio original, dado que el desarrollo se realiza de forma aislada sobre una bifurcación (*fork*). Sumado a esto, el uso de *Pull Requests* asegura que cada modificación sea revisada y validada por el mantenedor antes de su integración, incrementando la seguridad y estabilidad del código base. En cuanto a las dificultades, el modelo exige una curva de aprendizaje inicial más alta para gestionar correctamente los comandos de sincronización entre las distintas vías de los repositorios.

Conclusiones

Este proyecto permitió experimentar en la práctica lo que significa desarrollar de forma distribuida, entendiendo que colaborar en un mismo proyecto no se trata solo de escribir código funcional, sino de construir un proceso ordenado que permita a distintas personas intervenir sobre él sin comprometer su estabilidad. Trabajar desde un *fork*, manteniendo el enlace propio como *origin* y el del compañero como *upstream*, hizo posible desarrollar de forma autónoma mientras se sincronizaban periódicamente los cambios, evitando así que el trabajo individual afectara la rama principal antes de estar listo para integrarse.

Esta misma lógica se reforzó al trabajar en ramas independientes para cada nueva funcionalidad, lo que permitió avanzar en paralelo sin interferir con *main* hasta el momento de la integración final. El análisis del historial de *commits* confirmó que este proceso deja un registro trazable, donde cada cambio queda asociado a su autor, facilitando identificar con claridad el origen de cada aporte dentro del proyecto.

Todo este desarrollo partió de un issue bien definido, cuya descripción y justificación permitieron alinear las expectativas de manera clara antes de iniciar el trabajo, evitando ambigüedades sobre la contribución esperada. Posteriormente, la fase de revisión del Pull Request resultó ser una etapa clave. El intercambio de comentarios orientados a refinar el código demostró el valor de la revisión por pares para identificar detalles técnicos que suelen pasar desapercibidos. De esta forma, la aceptación del *Pull Request* funcionó como un verdadero filtro de validación técnica, asegurando la calidad del programa antes de su integración a la rama principal.

Para finalizar, la experiencia mostró que un flujo de trabajo colaborativo basado en este principio no solo protege el código, sino que ordena la colaboración misma, generando una buena trazabilidad, comunicación explícita y control de calidad en cada etapa del desarrollo conjunto.

Anexos

Enlaces al repositorio original

Fernando: <https://github.com/fernanaando/proyecto-repo>

Isidora: <https://github.com/printf-scarlettqrow/OptimizerProyect>

Enlaces a los *forks*

Fernando: <https://github.com/fernanaando/OptimizerProyect>

Isidora: <https://github.com/printf-scarlettqrow/proyecto-repo>

Enlaces a los *Pull Request* realizado por

Fernando: <https://github.com/printf-scarlettqrow/OptimizerProyect/pull/2>

Isidora: <https://github.com/fernanaando/proyecto-repo/pull/2>

Enlaces a los *issues* asociados

Fernando: <https://github.com/fernanaando/proyecto-repo/issues/1>

Isidora: <https://github.com/printf-scarlettqrow/OptimizerProyect/issues/1>

Captura de rama de contribución

1. Rama de Fernando en repositorio de Isidora

Your branches	
Branch	Updated
feature/exportar-json 	 3 days ago

2. Rama de Isidora en repositorio de Fernando

Your branches	
Branch	Updated
feature/clasificar-objeto-nuevo 	 2 days ago

Captura de cada *issue*

1. *Issue* de Fernando en repositorio de Isidora

nueva funcionalidad: exportación de las rutas calculadas a json #1

Closed 2

fernaando opened 1w ago Contributor

Explicación del problema y mejora:

hola, mirando tu repositorio del proyecto 1 me di cuenta de que los resultados quedan super bien en los mapas y gráficos de la carpeta /output pero siento que falta exportar los resultados de las rutas en un formato json para luego poder trabajar con ellos.

Justificación:

separar la visualización de los datos puros ayuda mucho porque por ejemplo después se podría pasar por otro sistema para ver las rutas exactas de cada entrega.

Descripción de los cambios esperados:

Hacer una función nueva que tome la ruta ya calculada y que genere un archivo .json dentro de la carpeta /output, el archivo tendría que llevar el orden exacto de los nodos que se visitaron y la distancia total.

Criterios de resolución:

el issue esta resuelto cuando al correr el programa se genere el archivo .json con los datos correctos en la carpeta /output.

2. *Issue* de Isidora en repositorio de Fernando

Agregar función de clasificación de nuevos objetos con validación de entradas #1

Closed 2

printf-scarlettrow opened 3d ago Last edited by printf-scarlettrow Contributor

Hola Fernando, estuve analizando tu repo y me gustaría comentar una mejora:

Explicación del problema y mejora

El modelo entrena y evalúa un RandomForestClassifier sobre el dataset, pero no existe ninguna forma de usar el modelo con un objeto nuevo que no esté en el CSV, y tampoco hay ningún mecanismo que evite pasarle datos mal formados, lo que podría generar predicciones sin sentido y sin notarlo.

Justificación

El modelo entrenado no tenía forma de usarse con datos nuevos, así se agrega valor práctico y maneja errores de entrada.

Descripción de los cambios esperados

Se propone agregar una función `clasificar_objeto` que reciba las seis características usadas por el modelo y valide que sean numéricas, lanzando `ValueError` claros en caso contrario, y que use `modelo.predict()` y `modelo.predict_proba()` para devolver la clase predicha junto con las probabilidades por clase, incluyendo pruebas con un caso válido y casos inválidos que demuestren el manejo de errores.

Criterios de resolución

El issue está resuelto cuando la función clasifica correctamente un objeto válido devolviendo clase y probabilidades, lanza `ValueError` con mensajes claros ante datos no numéricos, y el notebook incluye pruebas ejecutables que muestran ambos comportamientos sin que el kernel se caiga.

Commits realizados por cada contribuidor

1. Commits de Fernando en repositorio de Isidora

The screenshot shows the 'Commits' page for the repository 'feature/exportar...'. It displays a list of commits by user 'fernaaando'.

Commit Message	Commit Hash	Time Ago
Commits on Jul 17, 2026		
Agrega manejo de errores en controlador y test unitario para exportacion JSON	eade90d	3 days ago
Commits on Jul 15, 2026		
arreglo de errores	d7eb252	5 days ago
actualizacion de readme	5d0ced0	5 days ago
integracion de funcion json en controller.py para exportar la ruta calculada	1bcd74	5 days ago
agrega funcion para implementacion de exportacion json	3ff007d	5 days ago

2. Commits de Isidora en repositorio de Fernando

The screenshot shows the 'Commits' page for the repository 'feature/clarificar...'. It displays a list of commits by user 'printf-scarlettrow'.

Commit Message	Commit Hash	Time Ago
Commits on Jul 19, 2026		
Documentar función clasificar_objeto y mejorar legibilidad de probabilidades	9620b60	2 days ago
Commits on Jul 18, 2026		
Documenta la sección de clasificación de nuevos objetos	605ac5e	2 days ago
Agrega pruebas de clasificar_objeto con casos reales de cada clase	b38a355	2 days ago
Agrega función clasificar_objeto con validación de tipos de entrada	cf14c7a	2 days ago

Sección de *Files Changed* de cada *Pull Request*

1. Repositorio de Isidora:

The screenshot shows a GitHub Pull Request #2 titled 'Implementacion de exportación de resultados en formato JSON'. The 'Files changed' section is visible, showing changes to the file 'src/controller.py'.

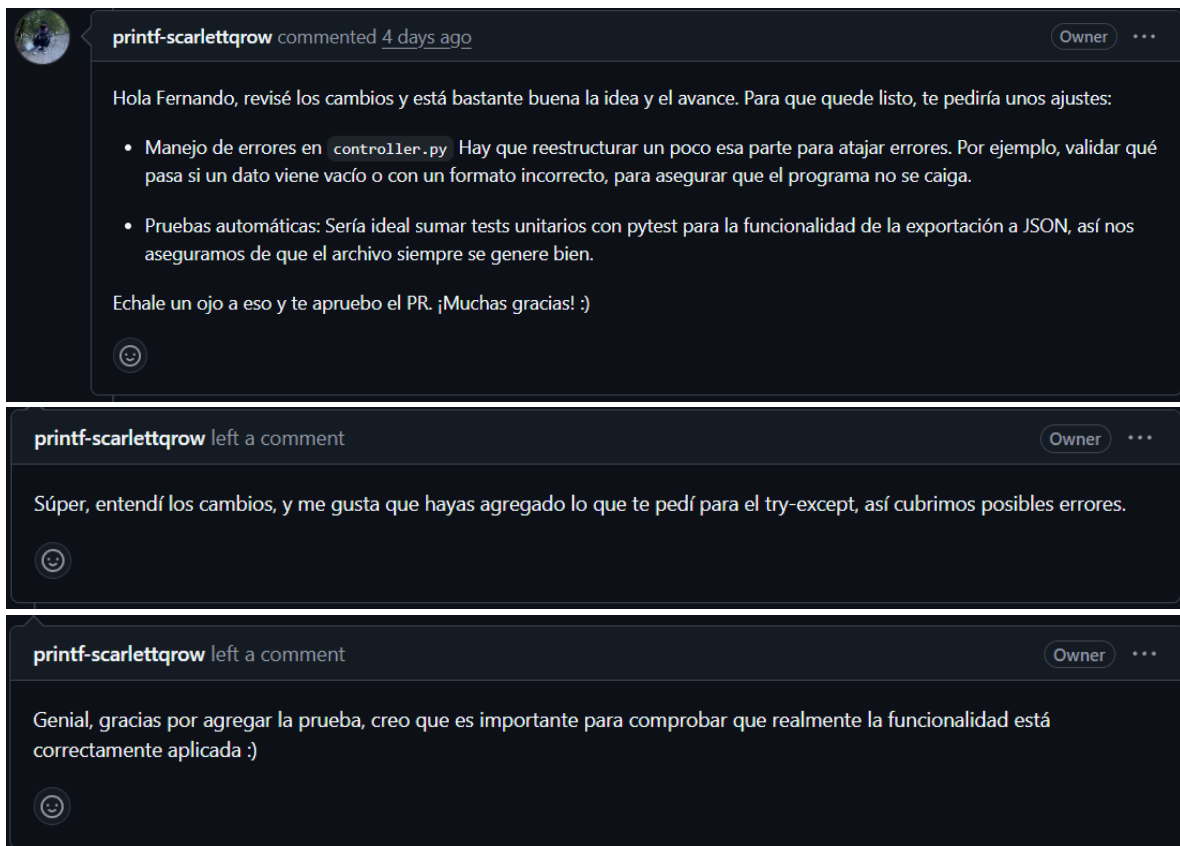
```
1  @staticmethod def export_route_to_json(route_id):
2  2  """Exporta la ruta calculada a un archivo JSON.
3  3  """
4  4  from .data_loader import DataLoader
5  5  from .route_calc import RouteCalculator
6  6  from .visualizer import RouteVisualizer
7  7
8  8  # Obtener la ruta calculada
9  9  route_calculator = RouteCalculator()
10 10  route_id = route_calculator.get_route_id()
11 11  # Obtener los datos de la ruta
12 12  data_loader = DataLoader()
13 13  route_data = data_loader.get_route_data(route_id)
14 14  # Guardar los datos en un archivo JSON
15 15  route_calculator.save_route_data(route_id, route_data)
16 16
17 17  # Crear el objeto de exportación
18 18  export_obj = ExportRouteToJson(route_id, route_data)
19 19  # Guardar el objeto en un archivo JSON
20 20  export_obj.save_data()
21 21
22 22  # Mostrar el mensaje de éxito
23 23  print(f"Ruta {route_id} exportada exitosamente.")
24 24
25 25  return route_id
```

2. Repositorio de Fernando:

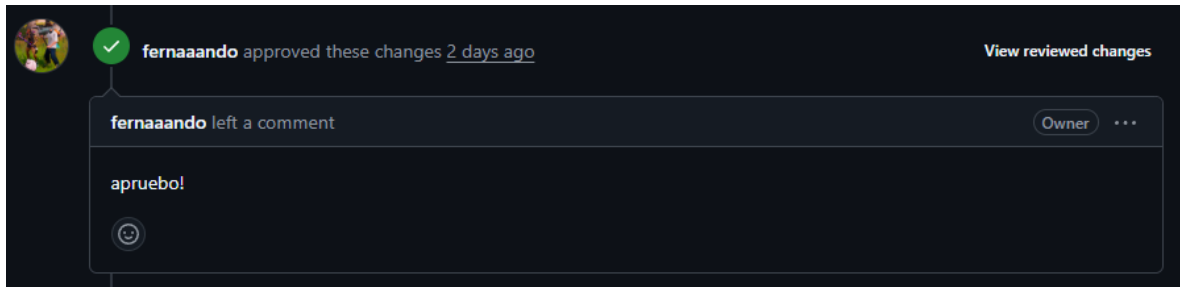
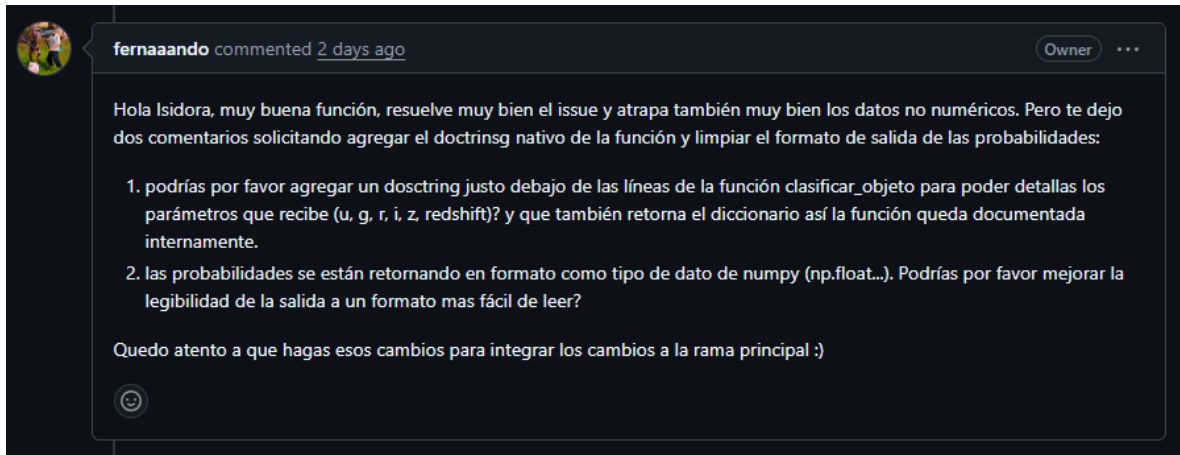


Captura de los comentarios de revisión

1. Comentarios de Isidora

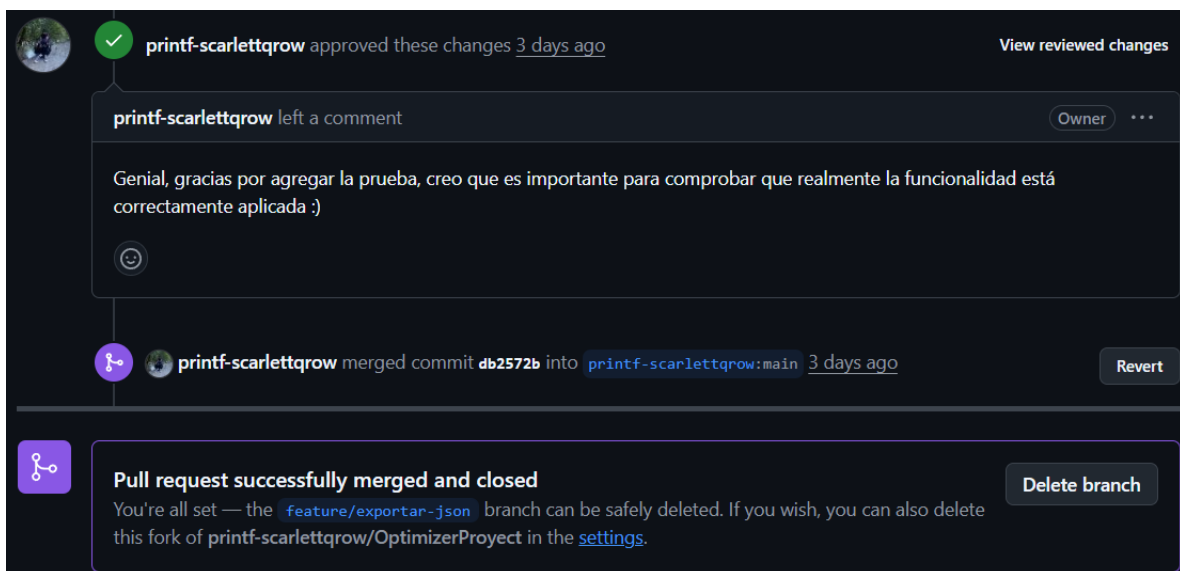


2. Comentarios de Fernando

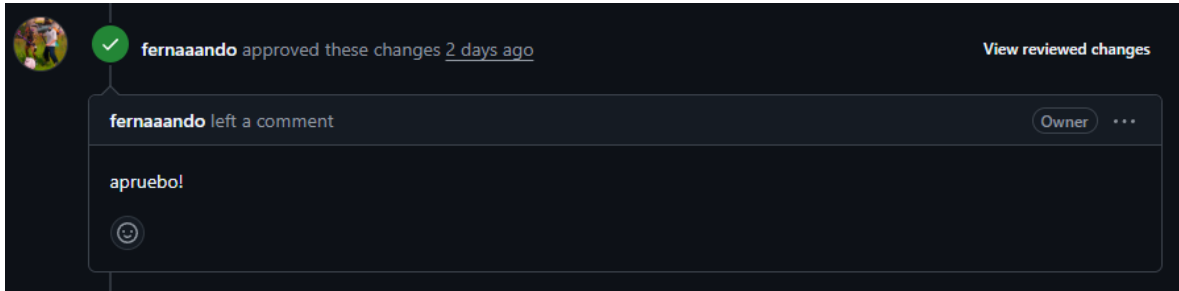


Evidencia de la aprobación del Pull Request

1. Aprobación de Isidora

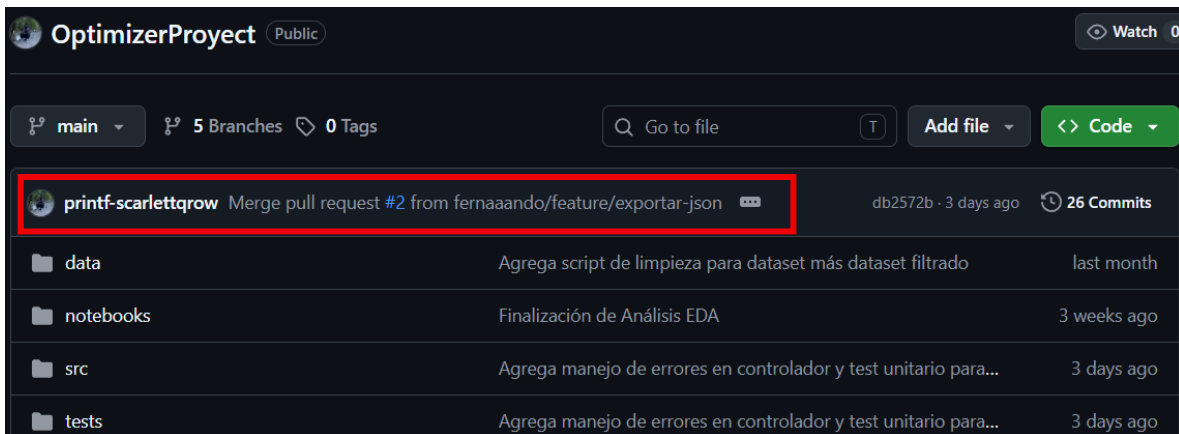


2. Aprobación de Fernando

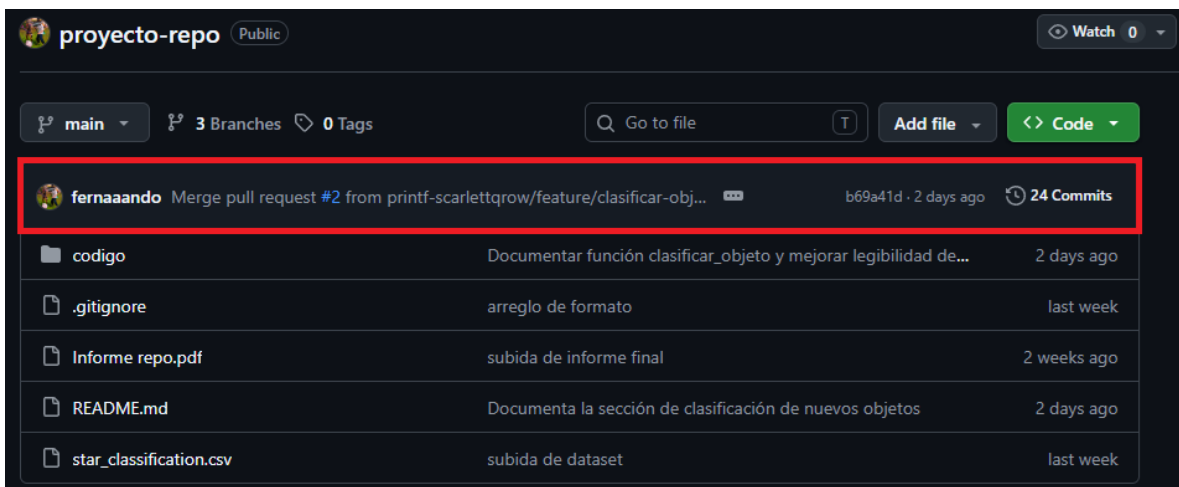


Evidencia del Pull Request Integrado

1. Repositorio de Isidora

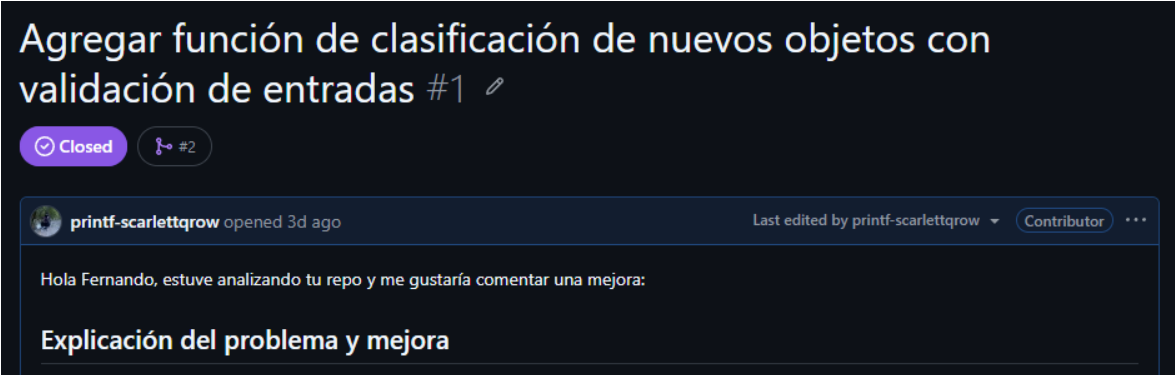


2. Repositorio de Fernando



Issue cerrado

1. Issue cerrado en repositorio de Fernando



2. Issue cerrado en repositorio de Isidora



Comando sobre cada repositorio

Fernando sobre el Fork de Isidora

Git remote -v:

```
(repo) fernando@fernando:~/s9/repo/OptimizerProyect$ git remote -v
origin  git@github.com:fernando/OptimizerProyect.git (fetch)
origin  git@github.com:fernando/OptimizerProyect.git (push)
upstream  git@github.com:printf-scarlettqrow/OptimizerProyect.git (fetch)
upstream  git@github.com:printf-scarlettqrow/OptimizerProyect.git (push)
```

Git branch --all:

```
(repo) fernando@fernando:~/s9/repo/OptimizerProyect$ git branch --all
* feature/exportar-json
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/exportar-json
  remotes/origin/main
  remotes/upstream/feature-EDA
  remotes/upstream/feature-algorithm
  remotes/upstream/feature-dataset
  remotes/upstream/feature-visualization
  remotes/upstream/main
```



Git log --graph --oneline --all:

```
(repo) fernando@fernando:~/s9/repo/OptimizerProject$ git log --graph --oneline --all
* eade90d (HEAD -> feature/exportar-json, origin/feature/exportar-json) Agrega manejo de errores en controlador y test unitario para exportacion JSON
* d7eb252 arreglo de errores
* 5d0ced0 actualizacion de readme
* 1bcdc74 integracion de funcion json en controller.py para exportar la ruta calculada
* 3ff007d agrega funcion para implementacion de exportacion json
* 374858b (upstream/main, origin/main, origin/HEAD, main) Agrega Informe Final
* 5acfd43 Fix en Controlador: Sobreescritura de Charts
* 74c1eb4 Fix: AlgorithmSolver y Agrega sección Interpretación a README
* 8b7867a Merge branch 'feature-visualization'
| \
| * e9f1368 (upstream/feature-visualization) Fix en archivos de src para visualización y control
|
|
* eade90d (HEAD -> feature/exportar-json, origin/feature/exportar-json) Agrega manejo de errores en controlador y test unitario para exportacion JSON
* d7eb252 arreglo de errores
```

Git log --pretty=format:"%h | %ad | %an | %ae | %s" --date=short

```
(repo) fernando@fernando:~/s9/repo/OptimizerProject$ git log --pretty=format:"%h | %ad | %an | %ae | %s" --date=short
eade90d | 2026-07-17 | fernando | fernando.palma@pregrado.uoh.cl | Agrega manejo de errores en controlador y test unitario para exportacion JSON
d7eb252 | 2026-07-15 | fernando | fernando.palma@pregrado.uoh.cl | arreglo de errores
5d0ced0 | 2026-07-15 | fernando | fernando.palma@pregrado.uoh.cl | actualizacion de readme
1bcdc74 | 2026-07-15 | fernando | fernando.palma@pregrado.uoh.cl | integracion de funcion json en controller.py para exportar la ruta calculada
3ff007d | 2026-07-15 | fernando | fernando.palma@pregrado.uoh.cl | agrega funcion para implementacion de exportacion json
374858b | 2026-07-05 | i z z i | izzisalgado@gmail.com | Agrega Informe Final
5acfd43 | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Fix en Controlador: Sobreescritura de Charts
74c1eb4 | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Fix: AlgorithmSolver y Agrega sección Interpretación a README
8b7867a | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Merge branch 'feature-visualization'
4a6f095 | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Merge branch 'feature-algorithm'
46a3b86 | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Actualización Final de README
e9f1368 | 2026-07-04 | printf-scarlettqrow | izzisalgado@gmail.com | Fix en archivos de src para visualización y control
19e3286 | 2026-07-02 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega controlador e iniciador __init__
cf20256 | 2026-07-02 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega visualizador y config
b479165 | 2026-06-30 | printf-scarlettqrow | izzisalgado@gmail.com | Implementación de Algoritmos de Cálculo de Distancias
78105aa | 2026-06-30 | printf-scarlettqrow | izzisalgado@gmail.com | Se implementan dataLoader y dataUtils
4e96b25 | 2026-06-29 | printf-scarlettqrow | izzisalgado@gmail.com | Finalización de Análisis EDA
f54ef05 | 2026-06-28 | printf-scarlettqrow | izzisalgado@gmail.com | Rellena inicialmente el notebook EDA
bf0bf00 | 2026-06-28 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega /notebooks y el notebook de Análisis Exploratorio (más una base de códigos sencilla)
1230491 | 2026-06-28 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega sección de Estructura de Repositorio a README
6d2bcba | 2026-06-28 | printf-scarlettqrow | izzisalgado@gmail.com | Fix: Mover scripts a /src y eliminar /codes
f53d8b8 | 2026-06-28 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega requirements.txt y reorden de carpetas
9459e87 | 2026-06-25 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega script de limpieza para dataset más dataset filtrado
7f24e3d | 2026-06-25 | printf-scarlettqrow | izzisalgado@gmail.com | Agrega README y Dataset
b21dd5d | 2026-06-25 | i z z i | izzisalgado@gmail.com | Initial commit
(repo) fernando@fernando:~/s9/repo/OptimizerProject$
```

Isidora sobre el Fork de Fernando

Git remote -v:

```
PS C:\Users\izzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo> git remote -v
origin git@github-scarlett:printf-scarlettqrow/proyecto-repo.git (fetch)
origin git@github-scarlett:printf-scarlettqrow/proyecto-repo.git (push)
upstream https://github.com/fernando/proyecto-repo.git (fetch)
upstream https://github.com/fernando/proyecto-repo.git (push)
PS C:\Users\izzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo>
```

Git branch --all:

```
PS C:\Users\izzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo> git branch --all
* feature/clasificar-objeto-nuevo
main
remotes/origin/HEAD -> origin/main
remotes/origin/feature/clasificar-objeto-nuevo
remotes/origin/main
remotes/upstream/HEAD -> upstream/main
remotes/upstream/knn
remotes/upstream/main
remotes/upstream/randomForest
PS C:\Users\izzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo>
```



Git log --graph --oneline --all:

```
* 9620b60 (HEAD -> feature/clasificar-objeto-nuevo, origin/feature/clasificar-objeto-nuevo) Documentar función clasificar_objeto y mejorar legibilidad de probabilidades
* 605ac5e Documenta la sección de clasificación de nuevos objetos
* b38a355 Agrega pruebas de clasificar_objeto con casos reales de cada clase
* cf14c7a Agrega función clasificar_objeto con validación de tipos de entrada
* 1641fad (upstream/main, upstream/HEAD, origin/main, origin/HEAD) arreglo de formato
* 36594f7 Subida de dataset
├── fcd0b9c Update README.md
├── 822f5cd subida de informe final
├── 7370472 (main) arreglo de formato
├── 9bc200e Update README.md
├── 2b586da subida de informe final
└── e9eab34 subida de dataset
├── 608debd estructura del proyecto
├── f5f7e7e instrucciones de uso README
├── 96af8b4 correccion de errores
├── 551ca8f merge con rama de clasificador knn
├── b79301a (upstream/knn) realizacion de segundo modelo de prediccion knn
├── 22fad39 merge 1
├── 936d7e2 (upstream/randomForest) implementacion de random forest y visualizacion con la matriz de confusion
├── 305f245 descripcion de cada variable a usar
├── 20b6bb2 finalizacion de preparacion de datos: division de datos train test
├── 286fb47 separacion de datos para predecir
├── 0844c67 analisis exploratorio de los datos (EDA)
├── 4a56b02 analisis exploratorio inicial
├── e714f3f configuracion inicial, agregar .gitignore y README
└── 8bfc4b1 Initial commit
```

Git log --pretty=format: "%h | %ad | %an | %ae | %s" --date=short

```
PS C:\Users\lizzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo> git log --pretty=format:"%h | %ad | %an | %ae | %s" --date=short
9620b60 | 2026-07-19 | printf-scarlettrow | lizzisalgado@gmail.com | Documentar función clasificar_objeto y mejorar legibilidad de probabilidades
605ac5e | 2026-07-18 | printf-scarlettrow | lizzisalgado@gmail.com | Documenta la sección de clasificación de nuevos objetos
b38a355 | 2026-07-18 | printf-scarlettrow | lizzisalgado@gmail.com | Agrega pruebas de clasificar_objeto con casos reales de cada clase
cf14c7a | 2026-07-18 | printf-scarlettrow | lizzisalgado@gmail.com | Agrega función clasificar_objeto con validación de tipos de entrada
1641fad | 2026-07-13 | fernando | fernando.palma@pregrado.uoh.cl | arreglo de formato
36594f7 | 2026-07-13 | fernando | fernando.palma@pregrado.uoh.cl | Subida de dataset
e9eab34 | 2026-07-13 | fernando | fernando.palma@pregrado.uoh.cl | subida de dataset
fcd0b9c | 2026-07-05 | Fernando | 147350417+fernaaando@users.noreply.github.com | Update README.md
822f5cd | 2026-07-05 | Fernando | 147350417+fernaaando@users.noreply.github.com | subida de informe final
608debd | 2026-07-05 | fernando | fernando.palma@pregrado.uoh.cl | estructura del proyecto
f5f7e7e | 2026-07-04 | Fernando | 147350417+fernaaando@users.noreply.github.com | instrucciones de uso README
96af8b4 | 2026-07-03 | fernando | fernando.palma@pregrado.uoh.cl | correccion de errores
551ca8f | 2026-07-03 | fernando | fernando.palma@pregrado.uoh.cl | merge con rama de clasificador knn
22fad39 | 2026-07-03 | fernando | fernando.palma@pregrado.uoh.cl | merge 1
b79301a | 2026-07-02 | fernando | fernando.palma@pregrado.uoh.cl | realizacion de segundo modelo de prediccion knn
305f245 | 2026-06-30 | fernando | fernando.palma@pregrado.uoh.cl | descripcion de cada variable a usar
936d7e2 | 2026-06-30 | fernando | fernando.palma@pregrado.uoh.cl | implementacion de random forest y visualizacion con la matriz de confusion
20b6bb2 | 2026-06-29 | fernando | fernando.palma@pregrado.uoh.cl | finalizacion de preparacion de datos: division de datos train test
286fb47 | 2026-06-29 | fernando | fernando.palma@pregrado.uoh.cl | separacion de datos para predecir
0844c67 | 2026-06-28 | fernando | fernando.palma@pregrado.uoh.cl | analisis exploratorio de los datos (EDA)
4a56b02 | 2026-06-26 | fernando | fernando.palma@pregrado.uoh.cl | analisis exploratorio inicial
e714f3f | 2026-06-26 | fernando | fernando.palma@pregrado.uoh.cl | configuracion inicial, agregar .gitignore y README
8bfc4b1 | 2026-06-26 | Fernando | 147350417+fernaaando@users.noreply.github.com | Initial commit
PS C:\Users\lizzis\OneDrive\Escritorio\Uni\Introducción a Repositorios\proyecto-repo>
```